Deep Learning for Time Series
Adapting MOIRAI for Time Series Classification

Auguste Vautrin [1 2] **Maxime Coppa** [1 2] **Marama Simoneau** [1 2] **Antoine Gilson** [1 2]

Abstract

In this work, we adapt MOIRAI, a pretrained universal forecasting Transformer, to multivariate time series classification on LSST by replacing its forecasting module with classification heads and studying pooling and fine-tuning strategies. We find that simple mean pooling is the most robust aggregation scheme, and that multi-scale patch representations consistently outperform single-scale ones. MOIRAI improves over standard deep baselines and frozen multi-scale representations are competitive with strong classical approaches. Overall, our results show that forecasting-pretrained time series foundation models transfer effectively to classification. Code, data, and model weights are available at https://github.com/Alpharama/DLforTimeSeries.

1. Introduction

Foundation models are now standard in language and vision, but their development for time series is more recent. In this work, we study the adaptation of a forecasting foundation model to time series classification. Specifically, we adapt MOIRAI (Woo et al., 2024), a universal forecasting Transformer, to multivariate classification on the LSST dataset. LSST is a time series classification benchmark derived from the PLAsTiCC challenge. It contains 2,459 training instances and 2,466 test instances, with 6 variables, 14 classes, and a fixed sequence length of 36, making it a multivariate multiclass classification task.

The **M**asked **E**nc**O**der-based **U**n**I**ve**R**s**A**l T**I**me Series Forecasting Transformer model (MOIRAI) (Woo et al., 2024) is a patch-based masked encoder that flattens multivariate inputs into a single sequence. It utilizes a multi-patch-size mechanism to handle heterogeneous temporal resolutions and Any-variate Attention to handle arbitrary numbers of variables. MOIRAI has been pretrained on the LOTSA dataset consiting over 27 billion observations accross 9 domains. MORAI is a natural backbone for our setting because it is explicitly designed for multivariate time series, and it is an open-source foundation model with strong forecasting performance.

Our goal is to propose a methodology for adapting MOIRAI to classification and to compare classification heads and fine-tuning regimes.

2. Method

2.1. Adapting MOIRAI for Classification

We adapt MOIRAI from a probabilistic forecasting model into a backbone for supervised time series classification. Our design principle is simple: we preserve the pretrained representation-learning components of the `uni2ts` architecture and replace the forecasting-specific output stage with classification-oriented modules.

More precisely, we retain the representation-learning part of the `uni2ts` model: input scaling, patch projection, mask token replacement, and the pretrained Transformer encoder. We remove the forecasting-specific decoding stage since our downstream task is classification rather than forecasting. After this modification, the model outputs a latent representations of size $[\texttt{combine_seq}, d_{\text{model}}]$, where $\texttt{combine_seq} = V \times N_{\text{patch}}$ and d_{model} is the hidden dimension. In all experiments, we use the `Small` version of MOIRAI, with $d_{\text{model}} = 384$, due to computational constraints.

V denotes the number of variables, $P \in \{8, 16, 32, 64\}$ the patch size, and N_{patch} the resulting number of non-overlapping temporal patches extracted from each variable over the input context window. The encoder produces a sequence of patch embeddings rather than a single representation. Since classification requires a fixed-dimensional input, we propose different techniques to aggregate these embeddings before the classification.

2.2. Masking Mechanism

MOIRAI is pretrained with a masked forecasting objective, where masked positions are replaced by learnable mask embeddings. We first investigate whether this masking mechanism remains useful for classification. We test four settings: standard masking with full pooling over all output tokens,

masking at the input but excluding mask-token outputs from the final pooling, removing mask tokens entirely from the input, and using only the output embeddings associated with the mask tokens. This allows us to determine whether mask tokens provide useful global information beyond their original forecasting role.

2.3. Pooling and Classification Heads

The encoder outputs a sequence of contextualized patch embeddings, which must be aggregated into a fixed-dimensional representation for classification.

Deterministic pooling. We first study simple aggregation baselines: flattening the sequence, and mean, max, and min pooling applied over patches, over variables, or globally.

Attention-based pooling. We then consider learned aggregation mechanisms across the patch dimension. Basic Attention computes a weighted average of patch embeddings using a learned context vector. Multi-Head Attention (MHA) extends this with multiple attention heads. Hierarchical MHA applies attention first over patches within each variable and then across variables. For these attention heads, the context vector can be either shared across variables or learned independently for each variable.

Classification heads. On top of the pooled representation, we use simple classifiers. For frozen-feature experiments, we use Ridge and Random Forest models. For end-to-end training, we use a linear classification layer.

2.4. Fine-Tuning Strategies

In all settings, the classification head is trained, and we vary how much of the pretrained backbone is fine-tuned. We compare four regimes: Frozen encoder, where only the classification head is optimized; Mask fine-tuning, where the head and the input mask embeddings are updated while the rest of the encoder remains frozen; LoRA fine-tuning, where the head and LoRA parameters inserted into the encoder's linear layers are optimized while the original backbone weights remain frozen; and Full fine-tuning, where the head and all encoder parameters are jointly optimized.

2.5. Selected Architectures.

Based on the preliminary experiments, we define four architectures. The first two are single-scale models using patch size 8, with either mean pooling or attention-based pooling. The next two are multi-scale models combining fine-resolution embeddings from patch size 8 and coarse-resolution embeddings from patch size 64. In the first multi-scale model, the independently pooled representations from both scales are concatenated before classification. In the second, patch-64 embeddings query patch-8 embeddings through cross-attention, and the resulting representation is fused with a residual connection. The number of attention heads and the fine-tuning regime are fixed from a preliminary study.

3. Experiments

3.1. Training and Evaluation

We evaluate all deep models on the LSST multivariate classification dataset. Because the dataset is class-imbalanced, we report both accuracy and macro recall. Models are optimized with AdamW, and we evaluate weight decay values of 0.01 and 0.05. Learning rates depend on the adaptation regime: 10^{-4} for frozen and mask fine-tuning, 5×10^{-5} for LoRA, and 10^{-5} for full fine-tuning.

Following (Feofanov et al., 2025), we use a 20% validation split for model selection. Early stopping with patience 5 is based on validation cross-entropy, which provides a more stable signal than accuracy on this relatively small validation set.

3.2. Baseline

As baselines, we consider one classical model and one deep model, both trained directly on LSST without pretraining. The classical baseline is a Random Forest trained on handcrafted features extracted from the raw series. These features include per-variable moments, extrema, temporal indices of minima and maxima, gradient-based statistics, and energy. The deep baseline is a 1D-CNN trained from scratch on the raw multivariate time series, with two temporal convolution layers followed by max-pooling and a linear classifier. The deep learning baseline reaches 0.60 test accuracy and 0.45 macro recall, and the Random Forest reaches 0.66 test accuracy and 0.48 macro recall.

3.3. Basic Pooling Study

To isolate the quality of pretrained representations, we evaluate basic pooling strategies on a frozen MOIRAI encoder with a Ridge Classifier. Aggregating across the patch dimension via mean pooling (0.64 accuracy) significantly outperforms variable dimension (0.59) and global pooling (0.60) (see Table 3). A Random Forest classifier yields lower overall performance than the linear Ridge model (max 0.60, Table 4), indicating that the frozen representations are already well structured

Beyond pooling dimensions, the masking configuration is crucial. Compared to the standard setup (0.64), ablating either mask inputs or outputs of the encoder drops accuracy to 0.62, while relying exclusively on mask outputs degrades it to 0.60 (Tables 5, 6 and 2). Building on these optimal

Table 1. Baselines and evaluation of selected architectures across 10 random seeds. P8 and P64 denote patch sizes 8 and 64.

Model	FT Regime	Accuracy	Macro Recall
Feature Engineering + Random Forest	–	0.66	0.480
Raw-series 1D-CNN	–	0.60	0.450
Frozen multi-scale MOIRAI + Ridge	Frozen (P8+P16+P32+P64)	**0.66**	–
Single-scale Mean Pooling	Full FT (P8)	0.643 ± 0.009	0.378
Single-scale MHA (4 heads)	Full FT (P8)	0.640 ± 0.007	0.373
Multi-scale Mean Pooling	Full FT (P8) + LoRA (P64)	$\mathbf{0.647 \pm 0.006}$	**0.380**
Multi-scale Cross-Attention	Full FT (P8) + LoRA (P64)	0.639 ± 0.011	0.372

configurations, concatenating pooled representations across multiple patch sizes reaches an accuracy of 0.66 with Ridge (Table 7), strictly outperforming all single patch size frozen baselines.

3.4. Advanced Pooling and Fine-tuning Study

We next attach a linear classification head and compare four adaptation regimes: frozen transfer, mask-only tuning, LoRA, and full fine-tuning. Both LoRA and full fine-tuning substantially improve performance over the frozen baseline, particularly when using mean pooling Figure 1. In contrast, mask-only tuning yields negligible gains. Notably, introducing parameter updates completely reverses the optimal temporal resolution: while the frozen encoder favors coarse grained representations (patch size 64), adaptive regimes excel with fine grained inputs (patch size 8) (Figure 2). Furthermore, among parameter-efficient methods, standard LoRA strictly outperforms AdaLoRA and DoRA (Figure 5).

Architecturally, simple mean pooling remains our most robust configuration, consistently outperforming attention-based aggregations in overall accuracy. Hierarchical MHA fails empirically, yielding scores predominantly below 0.60. Regarding standard MHA, fine-tuning shifts the optimal head count from many (frozen) to few (Figure 3). Interestingly, while MHA outperforms mean pooling when the encoder is frozen (Figure 1), this dynamic flips during fine-tuning. Once the encoder's internal attention weights are updated, simple mean pooling becomes the superior choice for overall accuracy. Nevertheless, MHA achieves highly competitive macro recall in these adaptive regimes (Figure 4), confirming its utility for isolating complex patterns in minority classes.

3.5. Evaluation of Selected Architectures

Concatenating basic pooled representations across multiple patch sizes on a frozen encoder achieved an accuracy of 0.66 with Ridge (see Table 7). This outperforms any single patch size baseline (see Table 3) and directly motivated our advanced multiple patch size architectures.

To ensure statistical robustness and mitigate initialization variance, the four defined model configurations (see Table 1) were rigorously evaluated. Each architecture was trained and tested across 10 independent runs using different random seeds.

As shown in Table 1, Multi-scale Mean Pooling achieves the highest accuracy (0.647). This confirms that aggregating multi-resolution patches improves performance and further highlights the robustness of simple mean pooling. Conversely, Multi-scale Cross-Attention fails to outperform its single-scale counterpart.

Per-class recall (Table 8) shows attention outperforms mean pooling on classes 0 and 1, while the reverse holds for 11 and 12. As well, in terms of macro recall, our fine-tuned models consistently underperform the baselines.

Notably, these fine-tuned architectures do not surpass our frozen multi-scale Ridge Classifier or the Random Forest baseline (both reaching 0.66). We attribute this to the small LSST training set, which makes deep fine-tuning prone to overfitting, as well as a hyperparameter tuning protocol that may lack the flexibility required to optimally train each specific architecture.

4. Conclusion

We show that MOIRAI transfers effectively from forecasting to multivariate time series classification on LSST. MOIRAI-based models outperform the deep baseline trained from scratch, and frozen multi-scale MOIRAI representations with mean pooling match the Random Forest feature-engineering baseline. Two design choices are consistently effective. First, simple mean pooling is the most robust aggregation strategy and outperforms attention-based alternatives across fine-tuning regimes. Second, multi-scale patch representations improve over single-scale ones, showing that combining temporal resolutions is beneficial for classification. At the same time, deep fine-tuning remains limited on LSST. Fine-tuned models do not surpass the strongest frozen-feature or feature-engineering baselines, which points to overfitting as the main bottleneck on this small dataset. A natural next step is to pretrain MOIRAI based classification models on broader time series classification dataset or to use pretraining objectives better aligned with discriminative tasks.

References

Feofanov, V., Wen, S., Alonso, M., Ilbert, R., Guo, H., Tiomoko, M., Pan, L., Zhang, J., and Redko, I. Mantis: Lightweight calibrated foundation model for user-friendly time series classification. *arXiv preprint arXiv:2502.15637*, 2025.

Woo, G., Liu, C., Kumar, A., Xiong, C., Savarese, S., and Sahoo, D. Unified training of universal time series forecasting transformers. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

A. Basic pooling table

Table 2. Accuracy (Ridge) Mask Only embeddings

Pooling (Mask Only)	**8**	**16**	**32**	**64**
flatten	0.59	0.59	0.60	0.60
mean over variates	0.57	0.56	0.57	0.58
max over variates	0.52	0.54	0.54	0.57
min over variates	0.54	0.54	0.54	0.56
mean max min over variates	0.57	0.57	0.57	0.58

Table 3. Accuracy (Ridge)

Pooling / Patch	**8**	**16**	**32**	**64**
flatten	0.61	0.63	0.62	0.63
global mean	0.59	0.59	0.58	0.60
global max	0.54	0.55	0.54	0.58
global min	0.55	0.55	0.54	0.57
global mean max min	0.58	0.58	0.58	0.59
mean over patches	0.62	0.62	0.62	**0.64**
max over patches	0.59	0.61	0.60	0.63
min over patches	0.59	0.61	0.60	0.63
mean max min over patches	0.61	0.63	0.63	0.63
mean over variates	0.59	0.59	0.58	0.59
max over variates	0.57	0.57	0.55	0.58
min over variates	0.58	0.57	0.55	0.58
mean max min over variates	0.60	0.59	0.57	0.59

Table 4. Accuracy (Random Forest)

Pooling / Patch	**8**	**16**	**32**	**64**
flatten	0.57	0.55	0.55	0.59
global mean	0.57	0.58	0.56	0.58
global max	0.55	0.52	0.52	0.56
global min	0.52	0.53	0.52	0.57
global mean max min	0.57	0.57	0.55	0.58
mean over patches	0.59	0.58	0.56	**0.60**
max over patches	0.57	0.57	0.55	0.60
min over patches	0.56	0.56	0.55	0.60
mean max min over patches	0.58	0.57	0.56	0.60
mean over variates	0.55	0.54	0.54	0.58
max over variates	0.53	0.51	0.51	0.57
min over variates	0.53	0.52	0.51	0.56
mean max min over variates	0.56	0.55	0.53	0.57

Table 5. Accuracy (Ridge) without using mask tokens output

Pooling Method	**8**	**16**	**32**	**64**
flatten	0.58	0.59	0.60	0.62
global mean	0.58	0.59	0.57	0.58
global max	0.54	0.55	0.53	0.57
global min	0.55	0.55	0.53	0.56
global mean max min	0.57	0.58	0.55	0.58
mean over patches	0.60	0.62	0.60	0.62
max over patches	0.59	0.60	0.60	0.62
min over patches	0.59	0.60	0.60	0.62
mean max min over patches	0.59	0.59	0.59	0.62
mean over variates	0.58	0.60	0.56	0.58
max over variates	0.57	0.57	0.55	0.57
min over variates	0.55	0.56	0.54	0.56
mean max min over variates	0.58	0.58	0.56	0.58

Table 6. Accuracy (Ridge) without using mask tokens as input

Pooling Method	**8**	**16**	**32**	**64**
flatten	0.58	0.59	0.61	0.62
global mean	0.59	0.59	0.58	0.59
global max	0.54	0.56	0.54	0.57
global min	0.55	0.55	0.54	0.57
global mean max min	0.58	0.59	0.57	0.58
mean over patches	0.61	0.60	0.61	0.62
max over patches	0.60	0.58	0.59	0.62
min over patches	0.60	0.59	0.59	0.62
mean max min over patches	0.62	0.58	0.59	0.60
mean over variates	0.58	0.59	0.59	0.59
max over variates	0.56	0.56	0.56	0.57
min over variates	0.56	0.57	0.57	0.57
mean max min over variates	0.57	0.57	0.59	0.58

Table 7. Accuracy - Multi-scale evaluation (patches 8+16+32+64 concatenated)

Pooling Method	**Ridge**	**Random Forest**
flatten	0.65	0.58
global mean	0.62	0.59
global max	0.59	0.56
global min	0.59	0.56
global mean max min	0.61	0.59
mean over patches	0.66	0.61
max over patches	0.66	0.59
min over patches	0.65	0.60
mean max min over patches	**0.66**	0.60
mean over variates	0.63	0.57
max over variates	0.62	0.55
min over variates	0.62	0.56
mean max min over variates	0.63	0.57

B. Heatmaps

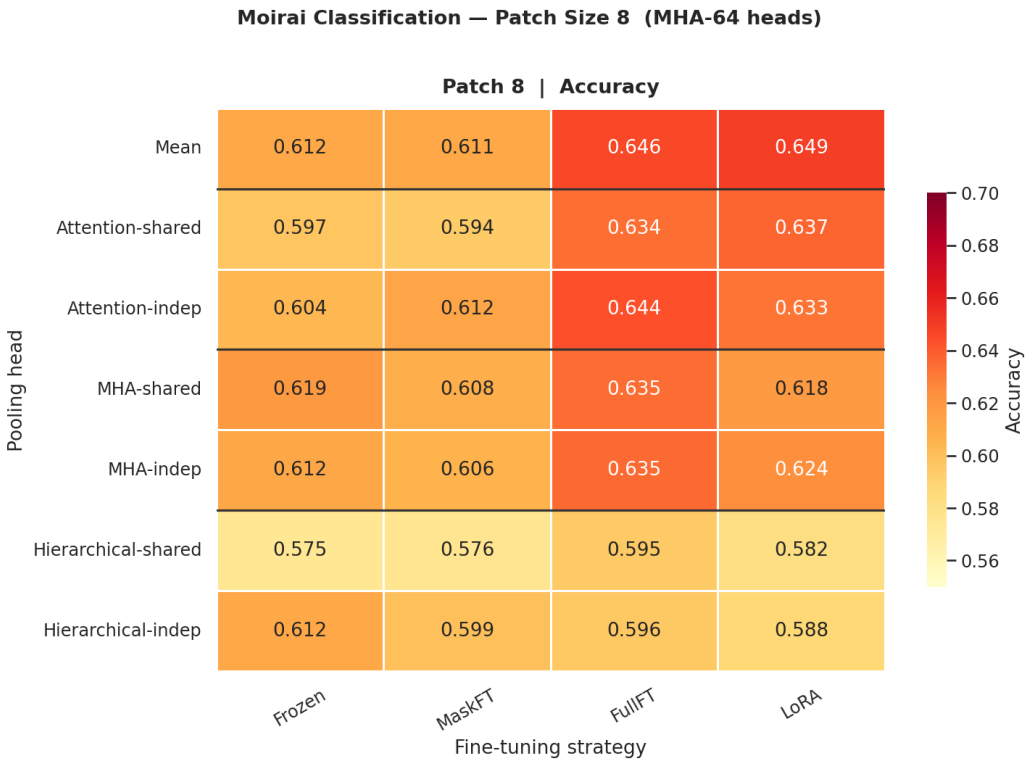

Figure 1. Classification accuracy (patch size 8) for each pooling head and fine-tuning strategy. MHA uses 64 heads.

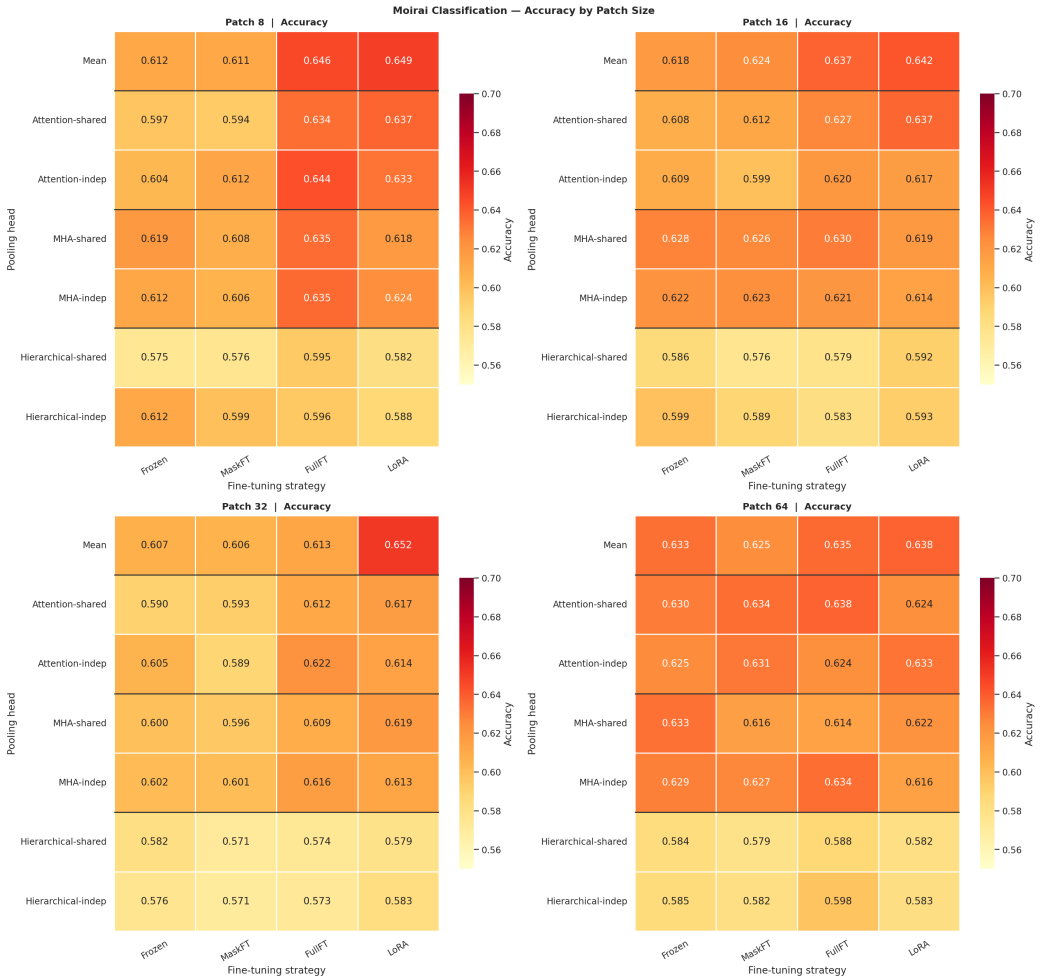

Figure 2. Classification accuracy across patch sizes {8, 16, 32, 64} for each pooling head and fine-tuning strategy.

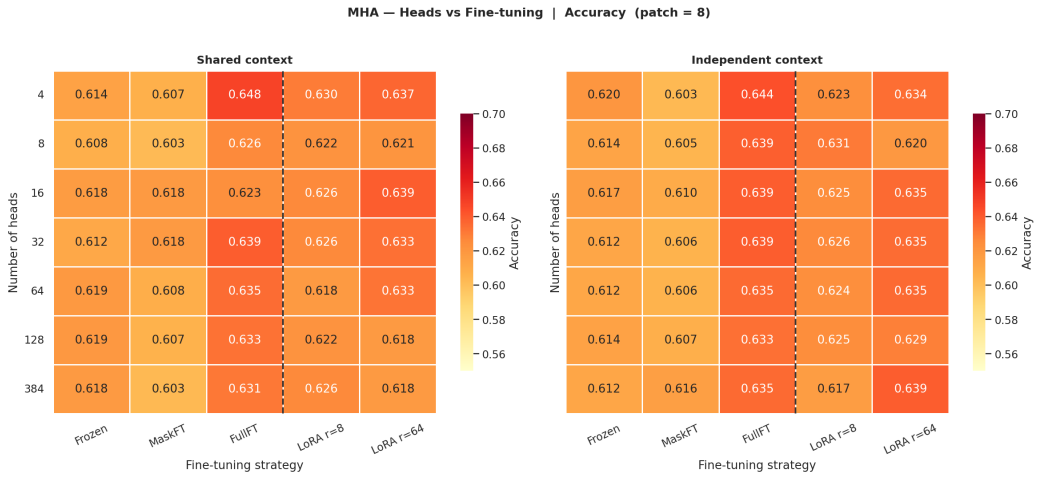

Figure 3. Accuracy as a function of the number of MHA heads (patch size 8), for each fine-tuning strategy and context mode (shared vs independent).

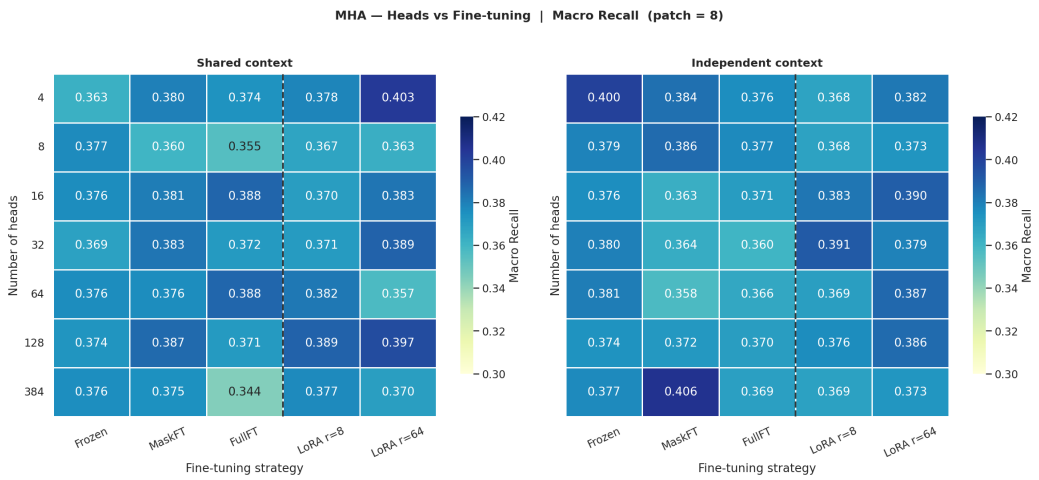

Figure 4. Macro recall as a function of the number of MHA heads (patch size 8), for each fine-tuning strategy and context mode.

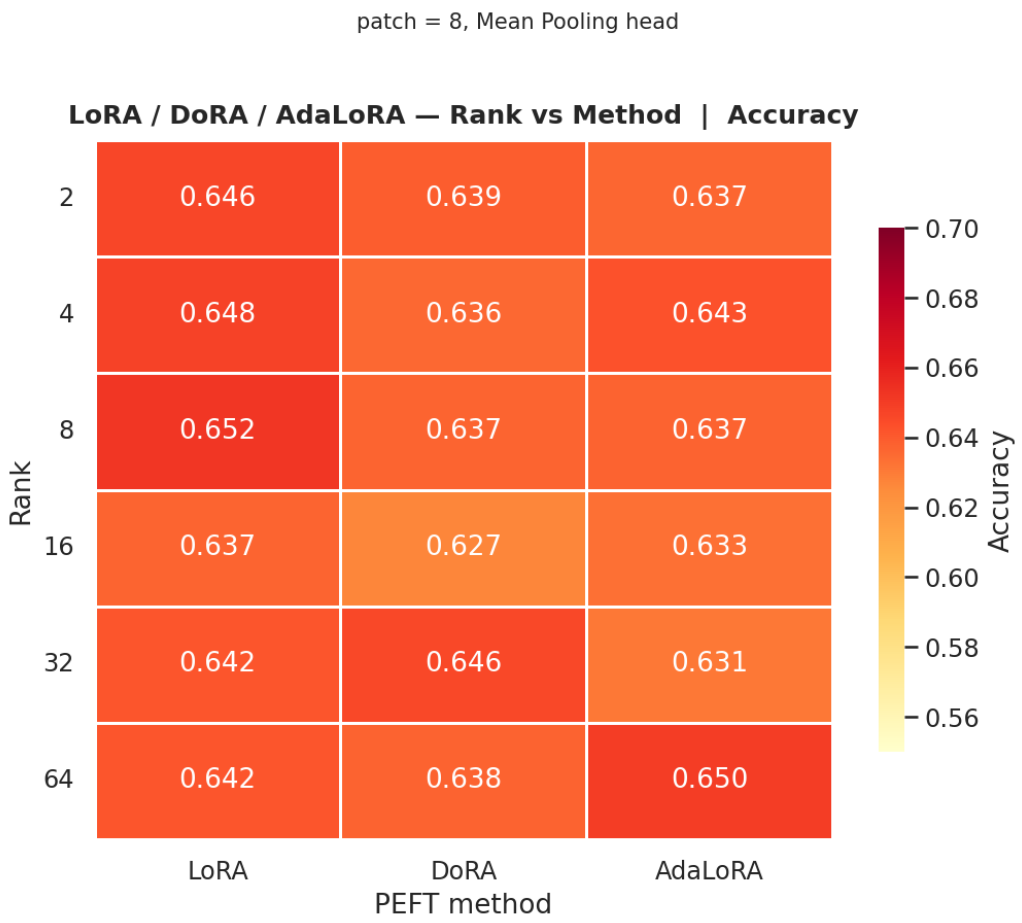

Figure 5. Accuracy of LoRA, DoRA, and AdaLoRA as a function of rank r (patch size 8, mean pooling head).

C. Class recall

Table 8. Per-class recall for the four final configurations. **N**: Number of instances (Support). **M1**: Mean Pooling (P8). **M2**: MHA (P8). **M3**: Multi-Scale Mean. **M4**: Multi-Scale Cross-Attention.

Class	N	M1	M2	M3	M4
0	124	0.36	**0.53**	0.27	**0.42**
1	270	0.93	**0.96**	0.89	**0.93**
2	382	0.29	0.35	0.44	0.38
3	63	0.00	0.00	0.00	0.00
4	7	0.00	0.00	0.00	0.00
5	35	0.03	0.00	0.00	0.00
6	153	0.15	0.14	0.01	0.15
7	24	0.00	0.00	0.00	0.00
8	313	0.91	0.89	0.87	0.87
9	68	0.00	0.00	0.00	0.01
10	121	0.98	0.93	0.97	0.98
11	777	**0.91**	0.86	**0.88**	0.86
12	77	**0.75**	0.45	**0.84**	0.57
13	52	0.13	0.13	0.12	0.13